

Code Overview

This project tackles **structured information extraction from doctor-patient dialogues** using a Llama model under four increasingly constrained decoding conditions (baseline → soft → medium → hard).

labeler/schema.py (The Canonical Data Schema)

This is the spine of the entire project. It defines the `ClinicalNote` Pydantic model, the structured output every component expects and produces. The fields are:

- **Scalar fields:** `name`, `age`, `duration` (null when absent)
- **List fields:** `symptoms`, `negated_symptoms`, `history`, `diagnosis`
- **Structured list:** `treatment` — each item is a `TreatmentItem` with a constrained `type` (one of `medication`, `test`, `referral`, `counseling`, `follow_up`, `procedure`, `equipment`) and a free-text `detail`

`schema.py` is important because it is *shared* by all three stages, labeling, inference, and evaluation, ensuring a consistent contract. In the medium and hard conditions, this Pydantic schema is also compiled into a JSON Schema that `lm-format-enforcer` uses at decode time to constrain token generation.

labeler/annotation_guide.py (Labeling Rules)

This defines `ANNOTATION_GUIDE`, a block of text injected verbatim into the GPT-4o labeling prompt. It's the authoritative specification for edge cases:

- Hedged endorsements ("I think I do", "a little") → go in `symptoms`, not `negated_symptoms`
- Lay → clinical term mapping ("trouble lying flat" → "orthopnea", "blood in urine" → "hematuria")
- History/diagnosis overlap is intentional — chronic conditions under active management appear in *both*
- Treatment items are broken out atomically (each medication action = separate entry)
- The source of truth is the **dialogue**, not the reference note (except for full names)

This file defines the annotation protocol.

labeler/few_shot_examples.py (Few-Shot Demonstrations)

Contains two fully hand-labeled encounters (D2N001 — Martha, an annual exam; D2N003 — John, an acute kidney stone case) used in:

- The **labeler** prompt (to teach GPT-4o the annotation style)
- The **soft, medium, and hard inference** prompts (to teach Llama the extraction format)

Critically, the file also exports `FEW_SHOT_IDS = {"D2N001", "D2N003"}`, which `run_inference.py` uses to **exclude these encounters from evaluation**, preventing data leakage.

labeler/labeler.py (Ground Truth Generation)

This script generates the ground-truth labels (`labels_clean.jsonl`) by calling GPT-4o on each dialogue.

- Uses **JSON mode** (`response_format={"type": "json_object"}`) to guarantee valid JSON from GPT-4o
- Validates each output against `ClinicalNote` via Pydantic; on failure, feeds the validation error back as a retry message (up to `MAX_RETRIES=2`)
- Writes labels incrementally as JSONL, so the job is **resumable** if it crashes
- The system prompt injects the full `ANNOTATION_GUIDE` and few-shot examples

labeler/spot_check.py (Label Quality Validation)

Compares LLM-generated labels against hand-labeled gold examples using the same set/F1 metrics as the main evaluator. This is how we measured **labeling noise**, the agreement between GPT-4o's labels and human annotations.

inference/schema.py (Inference-Side Schema)

A leaner copy of the schema used specifically by the inference and constrained decoding pipeline. In the hard condition, this schema gets dynamically extended with `Literal[...]` types for the ICD-10 vocabulary on `diagnosis` and `history` fields.

inference/llama_runner.py (The Core Experimental System)

It implements **LlamaRunner**, a class that loads Llama-3.2-3B-Instruct and generates structured JSON under four conditions:

Baseline: Minimal system prompt ("output JSON with these fields"), no few-shot, no constraints. Establishes the lower bound, i.e., what a model produces without any guidance.

Soft: A more detailed system prompt with explicit field-by-field instructions and few-shot examples. Tests whether better prompting alone improves extraction.

Medium: Same prompt as soft, but **lm-format-enforcer** enforces the JSON schema at decode time, guaranteeing structurally valid output token-by-token. Every generated token is filtered against what's allowed by the schema FSM.

Hard: Medium + the **diagnosis** and **history** fields are constrained to a **Literal** type built from ~thousands of ICD-10 long descriptions. This forces the model to select from a closed clinical vocabulary for these fields.

Important design decision: **symptoms are intentionally not vocabulary-constrained** in the hard condition. This is because using a vocabulary derived from our own gold labels would be circular, it would inflate scores by restricting the model to the exact answers we already have. There's no widely-used external symptom ontology (unlike ICD-10 for diagnoses), so symptoms remain free-text.

The file also includes **load_icd10_vocab()**, which reads the filtered ICD-10 CSV and returns sorted, deduplicated long descriptions as the hard-condition vocabulary.

inference/run_inference.py (Inference Orchestration)

The CLI script that runs all four conditions sequentially over the dataset. For each condition, it:

1. Loads ground-truth labels, strips out few-shot IDs (leakage prevention)
2. Instantiates **LlamaRunner** with the appropriate condition settings
3. Runs generation, validates each prediction against **ClinicalNote**, records latency
4. Writes **predictions_{condition}.jsonl** incrementally (resumable)

The output schema per record is **{encounter_id, prediction, valid, validation_error, latency_sec}**; the **valid** flag and **latency_sec** are used in the evaluator to compute validity rates and latency comparisons across conditions.

evaluation/evaluator.py (The Metrics Engine)

Two-layer evaluation system:

Deterministic metrics (run on every record, no API cost):

- `name_exact`, `age_exact`, `duration_exact` — exact match
- `set_f1()` — set-level precision/recall/F1 for `symptoms`, `negated_symptoms`, `history`, `diagnosis` (with `normalize_str` to handle casing, whitespace, punctuation defensively)
- `treatment_set_f1()` — compares treatments as `(type, normalized_detail)` tuples
- `hallucination_rate()` — fraction of predicted symptoms whose content words don't appear anywhere in the dialogue text (word-overlap grounding check)

LLM-as-judge (GPT-4o, optional, run on valid predictions only):

- Binary semantic equivalence per field (0/1)
- Grounding fraction for predicted symptoms and treatments (0.0–1.0)
- A one-sentence rationale per encounter

The LLM judge catches semantic equivalences that exact-match misses (e.g., "hematuria" vs. "blood in urine"). The judge's rationale strings also give us qualitative error analysis material.

`normalize_str()` and all F1 functions handle cases where the baseline produces wrong types (strings instead of lists, etc.), returning 0 rather than crashing.

evaluation/make_plots.py (Visualization)

Generates the six plots in `evaluation/Plots/` and `evaluation/plots_Alisa/` from `results_full.json`.

Summary: How the Files Connect

None

train.csv



labeler.py (GPT-4o + annotation_guide + few_shot_examples)



labels_clean.jsonl ←—— spot_check.py (quality validation)



run_inference.py (4 conditions)

└─ LlamaRunner [baseline] → predictions_baseline.jsonl

└─ LlamaRunner [soft] → predictions_soft.jsonl

└─ LlamaRunner [medium] → predictions_medium.jsonl (schema-constrained)

└─ LlamaRunner [hard] → predictions_hard.jsonl (schema + ICD-10 constrained)



evaluator.py

└─ deterministic metrics (all records)

└─ LLM judge (valid records only)



results_full.json → make_plots.py